

SLAP Bug Triage

Final Report

Mentor: Darshan

Period: May 2026 – July 2026

Repository: <https://github.com/angarajuNiveditha/slap-bug-triage>

Submitted: July 2026

Contents

1	Executive Summary	3
2	Problem Statement	3
2.1	The routing problem	3
2.2	Untapped signal in historical bugs	3
2.3	Media attachments	3
2.4	Scope	3
3	System Architecture	3
3.1	Overview	3
3.2	Repository layout	4
3.3	Pipelines	4
4	Data Layer	4
4.1	Jira integration	4
4.2	Building the embedding index	4
4.3	Class distribution	5
5	The Multi-Agent Pipeline	5
6	Media Sub-Agent	6
7	Component Classifier	6
7.1	Model	6
7.2	Hybrid fast / slow path	6
8	Similar-Bug Retrieval	6
8.1	Two-stage design	6
8.2	A concrete example	7
8.3	Merged retrieval and relevance gate	7
9	Dedup, Owner, and Triage	7
10	Architecture Skill Files	7
11	Streamlit UI and Local Ticket Store	8
11.1	Triage view	8
11.2	Local ticket store	9
11.3	Ticket Dashboard	9
12	Active-Learning Loop	9
13	Key Design Decisions	10
14	Future Work	10
14.1	Actual email input integration	10
14.2	Live Jira write integration	10
14.3	Full dashboard editing	10
14.4	Latency improvements	10
14.5	Configurable SLAP context	10
14.6	Multi-tenancy	11

15 Prototype to Production Mapping	11
16 Test Runs	11

1 Executive Summary

SLAP is Flipkart’s conversational shopping app where users chat with an AI assistant to discover products, try clothes virtually (VTON), and check out. Bug reports for SLAP can belong to any of five engineering teams — Backend, Backend-Labs, Data Science, UI, or Immersive — and routing them manually is slow and inconsistent.

This project builds a triage assistant that automates the first-pass workflow:

1. Parses a bug report (email or structured form, with optional screenshots or video).
2. Classifies the owning team using a local ML model, with Claude as a fallback for uncertain cases.
3. Finds the 10 most similar historical bugs from 564 real FLIPPI bugs.
4. Runs dedup, owner suggestion, and priority scoring in parallel via Claude sub-agents.
5. Produces a Jira-ready ticket draft. A human reviewer can override any field, then publish the ticket to a local dashboard.

2 Problem Statement

2.1 The routing problem

A bug titled “*the chat is broken*” could belong to any of five teams:

- **Backend** — the chat API returned an error or timed out
- **DS** — the model gave an irrelevant or wrong response
- **UI** — the chat screen crashed or froze on device
- **Backend-Labs** — the failure was in Styledrops or Social Finds
- **Immersive** — the failure was inside a 3D/AR view

Routing requires either domain knowledge or multiple Slack threads. The classifier handles the routine cases and flags genuinely ambiguous ones to a human.

2.2 Untapped signal in historical bugs

Every past FLIPPI bug is a labelled example: component, assignee, priority, description. Without automation, this signal is only used by engineers who happen to remember a similar case. The retrieval engine closes this loop.

2.3 Media attachments

Bug reports often include screenshots that contradict the text (e.g., “checkout is broken” with a login screen attached). Without automation, media must be manually inspected. The media sub-agent extracts structured findings and flags contradictions automatically.

2.4 Scope

The prototype does **not** auto-file tickets (a human must confirm every draft), does not handle production queuing or SLAs, and does not fine-tune any model.

3 System Architecture

3.1 Overview

The system has three layers:

Data layer Jira (read-only) + a one-time index builder that embeds 564 bugs and trains a classifier. The index is loaded from disk at startup — there is no live Jira fetch during triage.

Pipeline layer Three pipelines share the same index: multi-agent (Claude + Gemini, 40–180 s), rule-based (pure Python, 35 ms), and an SDK reference pipeline (blocked on corp network).

Presentation layer A Streamlit app with two views: a triage view (run pipeline, review, publish) and a dashboard (manage published BUGT-* tickets).

3.2 Repository layout

```
slap-bug-triage/
├── app.py                                # Streamlit UI
├── run_multi_agent.py                    # Multi-agent entry point
├── run_agent.py                          # Rule-based entry point
├── scripts/
│   ├── build_embedding_index.py
│   └── build_repo_skills.py
├── src/
│   ├── agents/                          # Multi-agent pipeline
│   │   ├── host_agent.py
│   │   ├── subagent_media.py
│   │   ├── subagent_parser.py
│   │   ├── subagent_form_consistency.py
│   │   ├── subagent_dedup.py
│   │   ├── subagent_owner.py
│   │   └── subagent_triage.py
│   ├── ml/                              # Local ML
│   │   ├── embedding_classifier.py
│   │   └── embedding_similarity.py
│   ├── rule_based/                      # Baseline pipeline
│   ├── shared/                          # Shared utilities
│   ├── sdk/                             # SDK reference (blocked)
│   └── slap_context/
│       ├── SLAP_KNOWLEDGE.md
│       ├── reference_screens/           # 16 Figma PNGs
│       └── architecture/               # Team skill files + per-repo files
├── data/
│   ├── bug_with_media/                  # 5 test bugs with media
│   ├── embedding_index.*                # gitignored, built locally
│   └── tickets.db                       # gitignored, local ticket store
```

3.3 Pipelines

Pipeline	Approach	Latency	Status
Multi-agent	Embedding classifier + Claude sub-agents + Gemini vision	40–180 s	Working
Rule-based	Regex parser + TF-IDF + keyword scorer. No LLM.	35 ms	Working
SDK reference	Anthropic Python SDK	—	Blocked (corp network)

4 Data Layer

4.1 Jira integration

`src/shared/jira_client.py` wraps Jira Cloud REST v3 with read-only access (search, get, list — no write methods). At runtime the pipeline uses Jira only to resolve metadata for the top-10 similar bugs (summary, assignee, priority, URL). The embedding index is pre-built and loaded from disk.

4.2 Building the embedding index

`scripts/build_embedding_index.py` runs once (or when new bugs are added):

1. Fetches up to 2000 component-labelled FLIPPI bugs from Jira.
2. Folds in human corrections from `data/corrections.csv`.
3. Encodes each bug (title + description) with `all-mpnet-base-v2` (768 dimensions).

4. Trains a Logistic Regression classifier on the resulting matrix.
5. Builds a team roster (component $\rightarrow$ engineers with ≥ 2 same-component assignments).
6. Writes three files to `data/` (all gitignored):

```
data/embedding_index.npz          # n 768 float32 + labels
data/embedding_index_logreg.pkl    # trained classifier
data/embedding_index_team_roster.json # component -> engineers
```

4.3 Class distribution

Component	Count	Notes
UI	~235	React Native + native iOS/Android
Backend	~200	API, auth, sessions, infra
Backend-Labs	~120	VTON, Styledrops, Social Finds
DS	~80	Ranking, model quality, relevance
Immersive	0	No labelled bugs in corpus

5 The Multi-Agent Pipeline

`src/agents/host_agent.py` (Astral) coordinates six sub-agents. Independent stages run in parallel via `ThreadPoolExecutor`. Each stage emits progress events that the Streamlit UI renders as a live stepper.

```
INPUT: email text / form + optional images / videos
|
+-- [1] subagent_media          (only if attachments)
|     Gemini describes each image in parallel
|     Claude maps descriptions to SLAP screens,
|     flags contradictions vs. email text
|     Fallback: Claude-direct vision if Gemini unavailable
|
+-- [2] subagent_parser         (Claude)
|     email + media findings -> structured BugReport
|
+-- [2b] subagent_form_consistency (only if form input)
|     flags mismatches between title / summary / steps
|
+-- [3a] EmbeddingClassifier    (local ML, ~7 ms)
|     LogReg on 768-dim embeddings
|     confidence >= 0.60: fast path
|     confidence < 0.60: Claude + skill files fallback
|
+-- [3b] EmbeddingSimilarityEngine (local ML)
|     cosine top-30 recall (~200 us)
|     cross-encoder rerank -> top-10 (~1.2 s)
|     searches Jira index + local ticket store
|
+-- [4] PARALLEL BLOCK (~8-15 s)
|     subagent_dedup    -> duplicate detection
|     subagent_owner    -> engineer / manager suggestion
|     subagent_triage   -> P0 / P1 / P2
|
+-- agent_ticket_builder -> Jira ADF draft + triage_notes JSON
|
+-- Human review (Streamlit)
|     Override any field -> saved to corrections.csv
|     Publish -> tickets.db + Dashboard
```

6 Media Sub-Agent

The original design had Claude analyse images directly. This was split into two stages.

Stage A (Gemini): Each image or video keyframe is described in plain prose by Gemini 2.5 Flash in parallel. Gemini has no SLAP context; it just describes what it sees.

Stage B (Claude): A single Claude call reads all Gemini descriptions together with `slap context` and 16 labelled reference screens, and produces structured `MediaFinding` JSON (screen name, anomalies, `contradicts_email_claim`).

Design decision: Two-stage vision

Gemini is faster for parallel raw vision. Claude is better at multi-step domain reasoning. Splitting the work lets each model do what it is best at. Claude-direct vision is preserved as a fallback if Gemini is unavailable.

Videos are handled by extracting up to 8 keyframes with `imageio-ffmpeg`. Videos longer than 60 s are rejected, prompting the reporter to trim.

Gotcha: Genvoy JWT expires hourly

The internal Gemini proxy requires a short-lived JWT that expires roughly every hour. The app shows a red banner when the token is expired, and `load_dotenv(override=True)` picks up a refreshed token from `.env` without a restart.

7 Component Classifier

7.1 Model

A Logistic Regression classifier trained on 768-dim sentence-transformer embeddings. Chosen over k-NN (no probability output) and small MLP (564 examples is too few). LogReg is interpretable, trains in seconds, and outputs calibrated probabilities needed for the confidence threshold.

```
LogisticRegression(class_weight="balanced", max_iter=2000, C=1.0)
# trained on (564, 768) matrix, 5 classes
```

7.2 Hybrid fast / slow path

```
HYBRID_CLAUDE_FALLBACK_THRESHOLD = 0.60

if max(proba) >= 0.60:
    return logreg_argmax          # ~7 ms
else:
    return claude_with_skills     # ~6 s, top-3 team skill files loaded
```

When LogReg is uncertain (below 60%), the top-3 candidate teams' architecture skill files are loaded into Claude's prompt so it can reason about which team actually owns the failure mode.

8 Similar-Bug Retrieval

8.1 Two-stage design

Stage 1 — cosine top-30 (~200 μ s):

```
sims = embeddings @ query # (n, 768) dot (768,)
top_idx = np.argmaxpartition(-sims, 30)[:30]
```

Fast but coarse — independent embeddings can miss paraphrases.

Stage 2 — cross-encoder rerank (~ 1.2 s): ms-marco-MiniLM-L-6-v2 takes both bug texts as joint input. Attention layers compare tokens across both texts, catching semantic matches that cosine similarity misses.

Design decision: Rerank top-30

Running the cross-encoder against all n candidates takes 5–10 s. Reranking only the cosine top-30 costs ~ 1.2 s with no meaningful quality loss.

8.2 A concrete example

Test bug: *App crashes on Proceed to Pay, Android 2.4.2.*

Rank	Bug	Cosine	After rerank
1	FLIPPI-616 (“Buy Now” tap crash)	0.685	0.411
2	FLIPPI-1198 (“continue to payment” crash)	0.624	0.790

Table 1: Cross-encoder correctly promotes FLIPPI-1198 by recognising “Proceed to Pay” and “continue to payment” as the same failure.

8.3 Merged retrieval and relevance gate

The engine searches both the Jira embedding index and the local ticket store (BUGT-* tickets published via the dashboard). Both lists are merged and reranked together so a recently published ticket can surface as a duplicate candidate even before it propagates to Jira.

When the top cross-encoder score is below 0.10, the result is flagged as low-confidence. Sub-agents fall back to reasoning from the bug text alone, and the UI shows an explanatory warning.

9 Dedup, Owner, and Triage

These three sub-agents run in parallel after classification and retrieval, cutting wall clock from ~ 25 s (serial) to ~ 10 s.

Deduplication. Claude reviews the top-10 similar bugs and raises a duplicate flag only when $\geq 80\%$ confident. False positives (blocking a new bug as a dup when it isn’t) are treated as worse than false negatives.

Owner suggestion. The escalation ladder, in order:

1. Non-manager engineer in the component roster who also appears on a similar bug.
2. If only managers match: the manager on the closest similar bug.
3. If no similar bugs exist: the component’s team manager from `TEAM_MANAGERS`.

The roster-membership check prevents an engineer from a different team from being suggested just because they once touched a same-component bug during a rotation.

Priority scoring. Similar-bug priorities are the primary signal. Hard overrides apply for 100%-reproducible crashes (P0) and security failures (auth infra, Grayskull $\rightarrow$ P0). P3 is excluded — vague reports go to a refile queue instead.

10 Architecture Skill Files

When LogReg confidence is below 0.60, Claude receives team-level and per-repo skill files in its prompt so it can reason about team ownership.

Team-level files (`slap_context/architecture/`) describe what each team owns, disambiguation rules between adjacent teams, and module-level ownership boundaries.

Per-repo files (slap_context/architecture/repos/) are auto-generated by scripts/build_repo_skills.py by mining cloned repos for: service class names, exception classes, HTTP entry points, Spring routes, and config files.(files for mozzarella and sphagetti were separately generated and added-due to access issues)

Design decision: Mine the code

The first version of skill files was Claude-drafted text. Reasoning was plausible but sometimes hallucinated class names. Regenerating from mined code names fixed this — every claim in a skill file is grep-verifiable in the actual codebase.

11 Streamlit UI and Local Ticket Store

11.1 Triage view

Slap Bug Triage

PIPELINE

In

Input

Email or structured form + optional screenshots / videos.

M

Media

Two-stage pass over attachments.

- Gemini **VISION** — per-image description in parallel.
- Claude **REASONING** — maps to SLAP screens + triage signals.

P

Parser

Turns text + media findings into a structured BugReport.

C

Classifier

Logging on 564 labelled bugs; falls back to Clauderskills when confidence < 0.60.

S

Similarity

Two-stage retrieval over historical FLIPPI tickets.

- Cosine **RECALL** — top-30 candidates in ~10 ms.
- Cross-encoder **RE-RANK** — resorts to top-10 using joint attention.

3 IN PARALLEL

D

Dedup

Duplicate decision over the top-5 (≥ 0.80 confidence).

O

Owner

Education ladder — first match wins.

- Engineer — with similar bug history on this team.
- Closest-bug manager — if only managers appear in similar bugs.
- Team manager — if no similar bugs on this component.

Dashboard (4)

STEP 1 · BUG REPORT

Input format

Email (.txt)

Structured form

Pipeline

Multi-agent (semantic, accepts images)

Rule-based (instant, text-only)

Bug title

The inline preference quiz that appears alongside chat results ("Quick one for you" bottomsheet) errors out when I tap an answer. After a noticeable delay with no visible resp

Platform

Android

Reporter

abc

Summary

Attached: bug_failed_to_load_preferences.mp4 — 28-second screen recording showing me on a chat results page (back-cover product cards visible at top: BRENNZ, OZMG), the "Quick one for you" sheet with the question "When trying new brands, what's your usual move?"

Steps to reproduce (one step per line — optional)

Steps to Reproduce:

- Open SLAP on Android, run any chat query that returns product cards (e.g. ask for "phone back covers")
- Wait for the "Quick one for you" preference bottomsheet to appear above the chat input

Attach screenshots or videos (multi-agent only)

screenshot.png

0.7MB

Your pick below will be saved to data/corrections.csv as a labelled training example for next time.

PRIORITY

P1

Critical

COMPONENT

Backend

Team: BE_Flippi

OWNER

Veeramreddy ChakradharReddy · Manager

DUPLICATE OF

BUGT-3

99% confidence

Open the Dashboard to inspect this local ticket.

Findings

Media findings

Field	Value
Component reasoning	LogReg was unsure — class probabilities were tight: UI 44% · Backend-Labs 23% · Backend 17% ← chosen · DS 16%. The classifier handed the decision to Claude, which reads the top-3 candidate teams' architecture skill files alongside the bug. Claude's reasoning: The bug is a backend error toast ('Failed to save preference') after a save attempt, meaning the server responded with a failure — not a rendering/interaction issue on the client. Per the Backend skill's disambiguation rule: 'If the service did not respond properly (5xx, timeout, null, wrong shape) → Backend.' Saving user preferences maps to user-memory/UserMemoryService in edison/. Nearest historical bugs: FLIPPI-475 (UI, sim 0.57), FLIPPI-1683 (Backend, sim 0.56), FLIPPI-1820 (DS, sim 0.49).
Triage signals	near-identical historical bug BUGT-3 filed P1, error toast on failed save, preference not saved but app still usable, all 5 similar bugs are P1
Severity justification	The nearest historical match (BUGT-3, similarity 0.999) is an essentially identical bug filed as P1, and all five similar bugs are unanimously P1. The issue is a 100%-reproducible backend error causing a toast notification and failed save — matching the P1 criteria for error messages and degraded (but not fully blocked) UX. No crash, payment, or secrets concern that would trigger a P0 override.
Owner reason	No engineer has been assigned a similar bug on this component. The closest similar bug (BUGT-3, sim 1.00) was owned by manager Veeramreddy ChakradharReddy — assigning to them as the most-relevant owner.

Similar Past Bugs

5 of 10 bugs cleared the relevance threshold (0.10). Only those 5 drove the owner + triage picks; the lower-scoring rows below are shown for context.

Key	Priority	Similarity	Assignee	Summary
BUGT-3	P1	0.999	Veeramreddy ChakradharReddy	[Chat/AI]: Preference-save API fails on 'Quick one for you' inline quiz — toast 'Failed to save preference. Please try again.'
FLIPPI-1556	P1	0.728	Kalnana Inn	There is no prompt/warning for allowed characters or L1 validation and its failure on save

8

The triage view provides: input toggle (email paste or structured form), pipeline toggle (multi-agent vs. rule-based), file uploader, a live progress stepper showing each pipeline stage in real time, and editable result tiles for Priority, Component, and Owner.

Notable UI behaviours:

- **Ambiguity banner:** shown when LogReg confidence is below 60%, displaying the full 5-class probability distribution.
- **Token-expiry banner:** red banner at the top of the page when the Genvoy JWT is expired or missing.
- **Refile banner:** shown when the report is too vague (missing 2+ required fields) or when an image contradicts the email text. The draft is withheld until the reporter provides more detail.
- **Overrides:** changes to Priority or Component are written to `data/corrections.csv` for the active-learning loop.
- **Publish:** saves the ticket to the local store and navigates to the dashboard.

11.2 Local ticket store

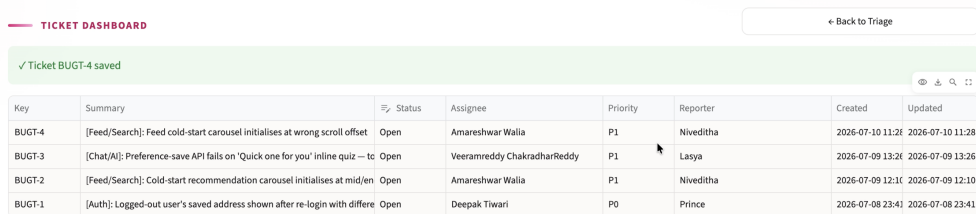
On Publish, `src/shared/db.py` (SQLAlchemy, SQLite or MySQL):

1. Inserts a row into the `tickets` table with all triage fields and an mpnet embedding.
2. Copies uploaded attachments to `data/tickets_attachments/<BUGT-N>/`.
3. Appends an INSERT event to `data/tickets_events.csv`.

The stored embedding enables local-ticket retrieval: future runs include BUGT-* tickets in similarity search alongside Jira bugs.

11.3 Ticket Dashboard

The dashboard is accessible from the **Dashboard (N)** button in the nav bar.



Key	Summary	Status	Assignee	Priority	Reporter	Created	Updated
BUGT-4	[Feed/Search]: Feed cold-start carousel initialises at wrong scroll offset	Open	Amreshwar Walla	P1	Niveditha	2026-07-10 11:26	2026-07-10 11:28
BUGT-3	[Chat/NI]: Preference-save API fails on 'Quick one for you' inline quiz — to	Open	Veeramreddy ChakradharReddy	P1	Lasya	2026-07-09 13:26	2026-07-09 13:26
BUGT-2	[Feed/Search]: Cold-start recommendation carousel initialises at mid/en	Open	Amreshwar Walla	P1	Niveditha	2026-07-09 12:10	2026-07-09 12:10
BUGT-1	[Auth]: Logged-out user's saved address shown after re-login with differe	Open	Deepak Tiwari	P0	Prince	2026-07-08 23:41	2026-07-08 23:41

12 Active-Learning Loop

Every Priority or Component override in the triage view is appended to `data/corrections.csv`. The next run of `scripts/build_embedding_index.py` folds these in as synthetic labelled bugs:

```
synthetic = {
    "key": f"CORRECTION-{row['bug_hash']}",
    "fields": {
        "summary": row["title"],
        "description": row["bug_text"],
        "components": [{"name": row["corrected_component"]}],
    },
}
```

Corrected labels are appended after the Jira fetch, so they override any conflicting Jira label. Corrections accumulate over time and improve the classifier without manual dataset curation.

13 Key Design Decisions

Read-only Jira. No write methods exist in `jira_client.py`. No accidental filings, no mis-routing, no RBAC complications.

Claude Code CLI instead of the Anthropic SDK. `console.anthropic.com` is unreachable on Flipkart's corp network. All LLM calls go through `claude -p` (headless CLI). Authentication is inherited from the local Claude Code session.

Gemini via the internal proxy. `generativelanguage.googleapis.com` is also blocked. `src/shared/genvoy_client.py` routes through Flipkart's internal proxy at `10.83.64.112` with dual-header auth (APIM key + Genvoy JWT). Falls back to Claude-direct vision on failure.

Two-stage retrieval. Cosine top-30 recall + cross-encoder rerank gives the same quality as a pure cross-encoder at ~ 1.4 s vs. 5–10 s.

Parallel sub-agents. Dedup, owner, and triage are independent. Running them in a thread pool saved 15–20 s per bug.

Three pipelines in one repo. The 35 ms rule-based pipeline provides a deterministic baseline for A/B comparison during tuning without needing any LLM.

14 Future Work

14.1 Actual email input integration

Currently bug reports are pasted as text into the Streamlit form. The production flow would be:

- Monitor the `fk-mart-ai-pulse` Gmail alias for incoming bug reports.
- Parse the email (subject, body, inline images, attachments) automatically via the Gmail API.
- Feed the parsed content directly into the triage pipeline without manual copy-paste.
- Reply to the reporter via SMTP with the triage result or a refile request.

14.2 Live Jira write integration

Wire the Publish button to the Jira write API (via MART MCP). On successful filing, store the returned Jira key (e.g. `FLIPPI-3210`) alongside the local `BUGT-N` key. Bidirectional sync would keep the dashboard status in sync with the actual Jira ticket lifecycle.

14.3 Full dashboard editing

Currently only Status is editable inline. Priority, Component, Assignee, and Summary should all be editable from the dashboard. Any component or priority change should feed into `corrections.csv` so dashboard edits contribute to the active-learning loop.

14.4 Latency improvements

Target: <30 s for text-only, <60 s for media.

- Stream partial triage results as sub-agents complete.
- Replace `ThreadPoolExecutor` for Gemini calls with `asyncio`.
- Cache media and similarity results for re-submissions with minor edits.
- Use a lighter model for the parser and form-consistency sub-agents.

14.5 Configurable SLAP context

The `slap_context/` directory is currently a static committed artifact. A settings panel in the Streamlit app could allow an admin to trigger a rebuild from source (Figma files, GitHub repos) without touching the CLI.

14.6 Multi-tenancy

The triage logic is product-agnostic; only the SLAP-specific assets are product-specific. Supporting other Flipkart products (Minutes, AI Mode, etc.) requires a per-tenant config file specifying the Jira project key, team labels, manager mapping, and skill-file paths.

15 Prototype to Production Mapping

Prototype	Production
Bug report text file	Gmail via <code>fk-mart-ai-pulse</code>
<code>run_multi_agent.py</code>	Astral agent runtime
<code>claude -p subprocess</code>	Envoy / FK-GPT (internal LLM gateway)
Gemini proxy at 10.83.64.112	Same endpoint (already production)
<code>data/embedding_index.npz</code>	Vector One (managed vector DB)
Jira REST v3 (read-only)	Jira via MART MCP (read + write)
Local <code>tickets.db</code>	MySQL on Flipkart PaaS

16 Test Runs

[Click here to watch the demo](#)